

Chat Context Export

Platform: ChatGPT

Exported: 13/09/2026, 15:27:02

Conversation ID: 6a9d3c42-faa8-83e8-b0ac-bbf496538ed9

Conversation

Assistant

Ah, yes — that's possible. The markers may not be visually obvious in your editor, especially if you're looking at a rendered/pretty view or the conflict was introduced somewhere you aren't currently looking. Let's let Git locate them exactly. From the repository root, run: If it shows lines For example: then we know exactly where they are. You can also use: If it shows nothing Then the error is coming from a different than the one you're looking at. Run: and: The error says: so CMake is specifically reading the at the directory you're configuring from. Most useful next step: run this and paste the output: Then I can pinpoint it immediately.

Code Block 1 (text)

```
git grep -n '<<<<<<\\|=====\\|>>>>>>'
```

Code Block 2 (text)

```
code/cpp/CMakeLists.txt:25:<<<<<< HEAD
code/cpp/CMakeLists.txt:40:=====
code/cpp/CMakeLists.txt:55:>>>>>> 7cb597c
```

Code Block 3 (text)

```
grep -n '<<<<<<\\|=====\\|>>>>>>' code/cpp/CMakeLists.txt
```

Code Block 4 (text)

```
pwd
find .. -name CMakeLists.txt -print
```

Code Block 5 (text)

```
cmake ..
```

Code Block 6 (text)

```
CMake Error at CMakeLists.txt:2
```

Code Block 7 (text)

```
git grep -n '<<<<<<\\|=====\\|>>>>>>'
```

User

```
(base) TLS-Fingerprinting git:(main) grep -n '<<<<<<\\|=====\\|>>>>>>' code/cpp/CMakeLists.txt
2:<<<<<< HEAD 14:===== 38:>>>>>> 7cb597c (Satyakam: Some CTests have been added) 51:<<<<<<
HEAD 58:===== 65:>>>>>> 7cb597c (Satyakam: Some CTests have been added) 74:<<<<<< HEAD
80:===== 108:>>>>>> 7cb597c (Satyakam: Some CTests have been added) 114:<<<<<< HEAD
124:===== 131:>>>>>> 7cb597c (Satyakam: Some CTests have been added) 136:<<<<<< HEAD
147:===== 200:>>>>>> 7cb597c (Satyakam: Some CTests have been added)
```

Assistant

Ah, now we know exactly what happened. They are there — they just weren't visible in the portion of the file you were looking at. You have 5 separate conflict regions in : And this explains why CMake fails at line 2. Don't manually guess which side to keep Because this is the conflict from your second commit: while your first commit has already been rebased successfully, the safest thing is to inspect the complete conflict. Run: This will print the entire file including the conflict markers. Paste that output here. I'll give you the exact final to use, preserving the remote changes and your CTest work, rather than telling you to blindly choose "ours" or "theirs." After we fix it, you'll do: So don't run — we want to preserve your second CTest commit.

Code Block 1 (text)

```
2-38
51-65
74-108
114-131
136-200
```

Code Block 2 (text)

```
Satyakam: Some CTests have been added
```

Code Block 3 (text)

```
sed -n '1,220p' code/cpp/CMakeLists.txt
```

Code Block 4 (text)

```
git add code/cpp/CMakeLists.txt
GIT_EDITOR=true git rebase --continue
```

User

```
(base) TLS-Fingerprinting git:(main) sed -n '1,220p' code/cpp/CMakeLists.txt
cmake_minimum_required(VERSION 3.14) <<<<<<< HEAD project(tlsfp LANGUAGES CXX) # Default to
Release build if not specified if(NOT CMAKE_BUILD_TYPE AND NOT CMAKE_CONFIGURATION_TYPES)
set(CMAKE_BUILD_TYPE Release CACHE STRING "Choose build type" FORCE) set_property(CACHE
CMAKE_BUILD_TYPE PROPERTY STRINGS "Debug" "Release" "RelWithDebInfo" "MinSizeRel") endif() #
Dependencies find_path(PCAP_INCLUDE_DIR NAMES pcap.h) find_library(PCAP_LIBRARY NAMES pcap)
===== project(tlsfp CXX) set(CMAKE_CXX_STANDARD 17) set(CMAKE_CXX_STANDARD_REQUIRED
ON) set(CMAKE_CXX_EXTENSIONS OFF) if(NOT CMAKE_BUILD_TYPE) set(CMAKE_BUILD_TYPE
Release) endif() # ----- # Dependencies #
----- find_path(PCAP_INCLUDE_DIR pcap.h)
find_library( PCAP_LIBRARY NAMES pcap wpcap ) >>>>>> 7cb597c (Satyakam: Some CTests have been
added) if(NOT PCAP_LIBRARY OR NOT PCAP_INCLUDE_DIR) message(FATAL_ERROR "libpcap/npcap
development files not found." ) endif() add_library(pcap::pcap UNKNOWN IMPORTED)
set_target_properties(pcap::pcap PROPERTIES IMPORTED_LOCATION "${PCAP_LIBRARY}"
INTERFACE_INCLUDE_DIRECTORIES "${PCAP_INCLUDE_DIR}" ) <<<<<<< HEAD find_package(OpenSSL
REQUIRED) find_package(Threads REQUIRED) # Executable Target add_executable(tlsfp_engine src/
main.cpp ===== # ----- # Static Library Target #
----- add_library(tlsfp_lib STATIC >>>>>> 7cb597c
(Satyakam: Some CTests have been added) src/capture.cpp src/db.cpp src/parser.cpp src/ja3.cpp src/
ja4.cpp src/db.cpp ) <<<<<<< HEAD target_compile_features(tlsfp_engine PRIVATE cxx_std_17) #
Compiler Warnings and Strict Dialect Enforcement if(CMAKE_CXX_COMPILER_ID MATCHES "GNU|Clang")
target_compile_options(tlsfp_engine PRIVATE ===== target_include_directories(tlsfp_lib PUBLIC
${CMAKE_CURRENT_SOURCE_DIR}/include ${PCAP_INCLUDE_DIR} ) target_link_libraries(tlsfp_lib PUBLIC
${PCAP_LIBRARY} OpenSSL::Crypto ) # ----- # Cross-
Platform Compiler Flags # ----- if(MSVC)
target_compile_options(tlsfp_lib PRIVATE /W4 /permissive- ) elseif(CMAKE_CXX_COMPILER_ID
MATCHES "GNU|Clang") target_compile_options(tlsfp_lib PRIVATE >>>>>> 7cb597c (Satyakam: Some
CTests have been added) -Wall -Wextra -Wpedantic -Wconversion -Wshadow <<<<<<< HEAD -Wnon-virtual-
dtor -Wold-style-cast -Wcast-align -Wunused -Woverloaded-virtual ${<<<CONFIG:Debug>:-O0 -g3 -
fsanitize=address,undefined> ) target_link_options(tlsfp_engine PRIVATE ${<<<CONFIG:Debug>:-
fsanitize=address,undefined> ===== ${<<<CONFIG:Release>:-O3> ${<<<CONFIG:Release>:-
march=native> ${<<<CONFIG:Debug>:-g> ${<<<CONFIG:Debug>:-O0> >>>>>> 7cb597c (Satyakam: Some
CTests have been added) ) endif() <<<<<<< HEAD target_include_directories(tlsfp_engine PRIVATE
include ) target_link_libraries(tlsfp_engine PRIVATE pcap::pcap OpenSSL::Crypto Threads::Threads )
===== # ----- # Main Binary Target #
----- add_executable(tlsfp_engine src/main.cpp )
target_link_libraries(tlsfp_engine PRIVATE tlsfp_lib ) # ----- #
CTest / GoogleTest # ----- include(CTest) if(BUILD_TESTING)
```

```
include(FetchContent) FetchContent_Declare( googletest GIT_REPOSITORY https://github.com/google/
googletest.git GIT_TAG release-1.12.1 ) FetchContent_MakeAvailable(googletest) # Unit tests
add_executable(tlsfp_unit_tests tests/test_fingerprints.cpp ) target_link_libraries(tlsfp_unit_tests PRIVATE
tlsfp_lib GTest::gtest_main ) # Register GoogleTest tests with CTest include(GoogleTest)
gtest_discover_tests(tlsfp_unit_tests) endif() >>>>>> 7cb597c (Satyakam: Some CTests have been
added)
```

Assistant

Yes. Now we can resolve this cleanly. You have two versions being combined: HEAD = the remote/current version. = your CTest commit. We want the library + CTest structure from your commit, while preserving useful improvements from the remote version. There is also a duplicate in the conflict; we should remove it. The clean resolved should be: Resolved CMakeLists.txt What I changed The important part is that we're not simply choosing one side. We kept: static library structure from your CTest refactor. linking against . GoogleTest/CTest setup. . The stronger compiler warnings from the remote version. and . Cross-platform lookup with /. The Debug sanitizer configuration. And removed: duplicate. Now replace the entire file Since there are five conflict regions, don't try to edit each marker individually. Just replace the entire contents of: with the resolved version above. Then verify there are no conflict markers: It should print nothing. Then: If the rebase finishes, then build/test: One thing to note: because you're currently on with a clean tree after the previous rebase step, this second conflict is still part of the rebase state even though your prompt shows . The is still the correct command.

Code Block 1 (text)

```
cmake_minimum_required(VERSION 3.14)

project(tlsfp CXX)

# -----
# Build configuration
# -----

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)

if(NOT CMAKE_BUILD_TYPE AND NOT CMAKE_CONFIGURATION_TYPES)
set(CMAKE_BUILD_TYPE Release CACHE STRING "Choose build type" FORCE)
set_property(CACHE CMAKE_BUILD_TYPE PROPERTY STRINGS
"Debug"
"Release"
"RelWithDebInfo"
"MinSizeRel"
)
endif()
```

```
# -----  
# Dependencies  
# -----
```

```
find_path(PCAP_INCLUDE_DIR NAMES pcap.h)
```

```
find_library(  
PCAP_LIBRARY  
NAMES pcap wpcap  
)
```

```
if(NOT PCAP_LIBRARY OR NOT PCAP_INCLUDE_DIR)  
message(FATAL_ERROR  
"libpcap/npcap development files not found."  
)  
endif()
```

```
add_library(pcap::pcap UNKNOWN IMPORTED)
```

```
set_target_properties(pcap::pcap PROPERTIES  
IMPORTED_LOCATION "${PCAP_LIBRARY}"  
INTERFACE_INCLUDE_DIRECTORIES "${PCAP_INCLUDE_DIR}"  
)
```

```
find_package(OpenSSL REQUIRED)  
find_package(Threads REQUIRED)
```

```
# -----  
# Static Library Target  
# -----
```

```
add_library(tlsfp_lib STATIC  
src/capture.cpp  
src/db.cpp  
src/parser.cpp  
src/ja3.cpp  
src/ja4.cpp  
)
```

```
target_include_directories(tlsfp_lib  
PUBLIC  
${CMAKE_CURRENT_SOURCE_DIR}/include  
${PCAP_INCLUDE_DIR}  
)
```

```
target_link_libraries(tlsfp_lib  
PUBLIC  
pcap::pcap  
OpenSSL::Crypto  
Threads::Threads  
)
```

```

# -----
# Compiler flags
# -----

if(MSVC)

target_compile_options(tlsfp_lib PRIVATE
/W4
/permissive-
)

elseif(CMAKE_CXX_COMPILER_ID MATCHES "GNU|Clang")

target_compile_options(tlsfp_lib PRIVATE
-Wall
-Wextra
-Wpedantic
-Wconversion
-Wshadow
-Wnon-virtual-dtor
-Wold-style-cast
-Wcast-align
-Wunused
-Woverloaded-virtual

$<$<CONFIG:Release>:-O3>
$<$<CONFIG:Release>:-march=native>

$<$<CONFIG:Debug>:-O0>
$<$<CONFIG:Debug>:-g3>
$<$<CONFIG:Debug>:-fsanitize=address,undefined>
)

target_link_options(tlsfp_lib PRIVATE
$<$<CONFIG:Debug>:-fsanitize=address,undefined>
)

endif()

# -----
# Main Binary Target
# -----

add_executable(tlsfp_engine
src/main.cpp
)

target_link_libraries(tlsfp_engine
PRIVATE
tlsfp_lib
)

```

```

# -----
# CTest / GoogleTest
# -----

include(CTest)

if(BUILD_TESTING)

include(FetchContent)

FetchContent_Declare(
  googletest
  GIT_REPOSITORY https://github.com/google/googletest.git
  GIT_TAG release-1.12.1
)

FetchContent_MakeAvailable(googletest)


# Unit tests
add_executable(tlsfp_unit_tests
  tests/test_fingerprints.cpp
)

target_link_libraries(tlsfp_unit_tests
  PRIVATE
  tlsfp_lib
  GTest::gtest_main
)


# Register GoogleTest tests with CTest
include(GoogleTest)

gtest_discover_tests(tlsfp_unit_tests)

endif()

```

Code Block 2 (text)

```
src/db.cpp
```

Code Block 3 (text)

```
code/cpp/CMakeLists.txt
```

Code Block 4 (text)

```
grep -n '<<<<<<\\|=====\\|>>>>>>' code/cpp/CMakeLists.txt
```

Code Block 5 (text)

```
git add code/cpp/CMakeLists.txt  
GIT_EDITOR=true git rebase --continue
```

Code Block 6 (text)

```
cd code/cpp/build  
cmake ..  
cmake --build . -j  
ctest --output-on-failure
```

Exported by Chat Context Manager